



Plan du cours



- **Chap 1 : Initiation au Langage Java**
 - Introduction
 - Outils de développement
 - Données, Types et Operateurs
 - Les structures alternatives et répétitives
 - Les tableaux de données et chaînes de caractères
 - Les instructions d'écriture et de lecture
- **Chap 2 : Java Orienté Objet**
 - **Classes et Objets**
 - **Héritage**
 - **Packages**
 - **Classes internes et Interfaces**
 - **Les exceptions**
- **Chap 3 : Introduction aux IHMs**
 - Ergonomie
 - Le patron d'architecture logicielle MVC
 - Evolution des IHM dans le temps
- **Chap 4 : Eléments de base d'une IHM Java**
 - Composants de base
 - Composants SWING
 - Les conteneurs
 - Les composants atomiques
 - Evènements & Ecouteurs
 - Principes programmation événementielle
 - Listeners java
 - Gestionnaires de positionnement
- **Chap 5 : Connexion à une base de données**

IHM - Java

2

Java Orienté Objet

(Chapitre 2)



Le concept de classe :

Une **classe** est un ensemble de données et de fonctions regroupées dans une même entité pour former un type d'objets donné.

Dans une classe :

- Les **données** sont appelées **attributs** ou **propriétés**.
- Les **fonctions** opérant sur les données sont appelées des **méthodes** ou **comportements**.

Pour accéder à une classe il faut en déclarer une **instance** de classe ou **objet**.

→ **Instancier** une classe consiste à **créer** un objet sur son modèle.

→ Entre classe et objet il y a, en quelque sorte, le même rapport qu'entre type et variable

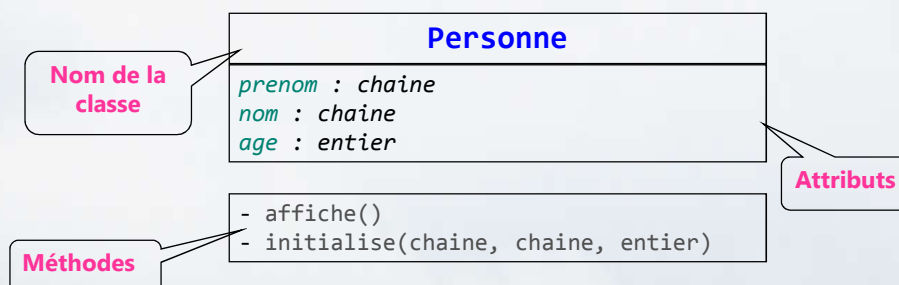
Java Orienté Objet

(Chapitre 2)



Le concept de classe :

Exemple : Classe Personne.



Java Orienté Objet

(Chapitre 2)



Le concept de classe :

Codage de la classe « Personne » en Java

```
public class Personne{

    private String prenom;
    private String nom;
    private int age;

    public void initialise(String P, String N, int age){
        prenom=P;
        nom=N;
        this.age=age;
    }

    public void affiche(){
        System.out.println(prenom+" "+nom+" "+age); }
    }
}
```

Attributs

Méthodes

Nom classe

IHM - Java

5

Java Orienté Objet

(Chapitre 2)



Forme générale d'une classe:

```
modificateur_accès class C1{
    type1 p1; // propriété p1
    type2 p2; // propriété p2
    ...
    type3 m3(...){ // méthode m3
        ...
    }
    type4 m4(...){ // méthode m4
        ...
    }
    ...
}
```

A partir de la classe C1, on peut créer de nombreux objets O1, O2,...

IHM - Java

6

Java Orienté Objet

(Chapitre 2)



Classe et visibilité des attributs :

Caractéristiques d'un attribut :

- Variable « globale » de la classe
- Accessible dans toutes les méthodes de la classe

```
public class Personne{
    private String prenom;
    private String nom;
    private int age;
    public void initialise(String P, String N, int age){
        this.prenom=P;
        this.nom=N;
        this.age=age; }
    public void affiche(){
        System.out.println(prenom+", "+nom+", "+age); }
}
```

Attribut visible dans les méthodes

N.B. Ne pas confondre attribut et variable d'une méthode

IHM - Java

7

Java Orienté Objet

(Chapitre 2)



Les objets :

Un **objet** est une **instance** d'une seule classe

- Se conforme à la description que sa classe fournit
- Admet une valeur propre à lui pour chaque attribut déclaré dans la classe
- Les valeurs des attributs caractérisent l'**état** de l'objet
- Possibilité de lui appliquer toute opération (**méthode**) définie dans la classe

Tout objet est manipulé et identifié par sa **référence**

- On parle indifféremment d'**instance**, de **référence** ou d'**objet**

IHM - Java

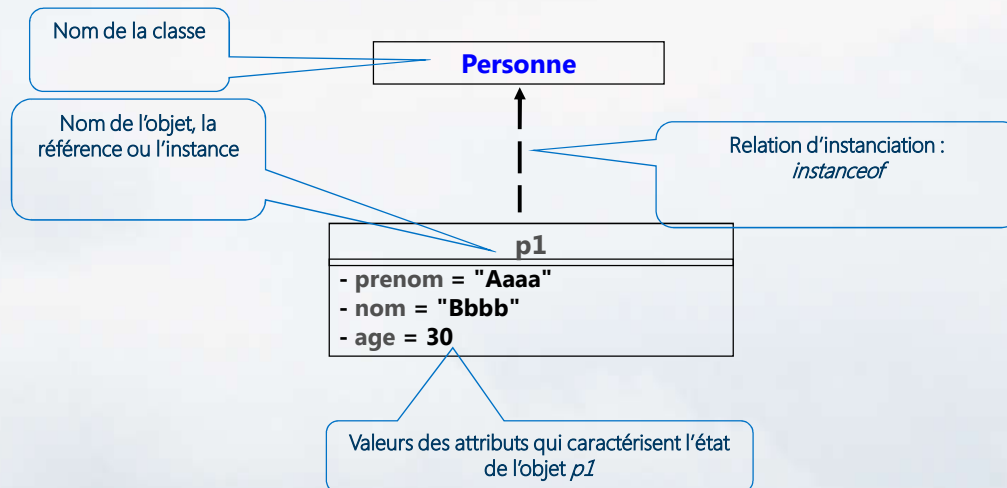
8

Java Orienté Objet

(Chapitre 2)



Exemple : Objet de type Personne :



IHM - Java

9

Java Orienté Objet

(Chapitre 2)



Création d'un objet :

Instancier la classe de cet objet
→ L'opérateur **new**

Syntaxe de déclaration `nom_classe nom_objet;`

Syntaxe de création `nom_objet = new nom_classe ();`

Exemple : création d'un objet de la classe Personne

```

public class Test1{
    public static void main(String arg[]){
        ...
        Personne p1=new Personne();
        ...
    }
}
  
```

IHM - Java

10

Java Orienté Objet

(Chapitre 2)

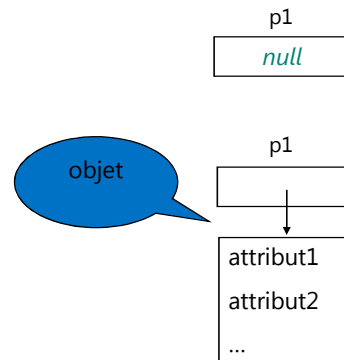
Déroulement de la création d'objets

Personne p1=new Personne();



1. Personne p1;

2. p1=new Personne();



IHM - Java

11

Java Orienté Objet

(Chapitre 2)



Programme de test :

```
// import Personne;
public class Test1{
    public static void main(String arg[]){
        Personne p1=new Personne ();
        p1.initialise ("Aaaa","Bbbb",30);
        System.out.print("p1=");
        p1.affiche();
    }
}
```

Résultat

p1= Aaaa , Bbbb , 30

IHM - Java

12

Java Orienté Objet

(Chapitre 2)



Remarques :

- 1- la variable p1 contient une **référence** (adresse) sur l'objet instancié.
- 2- L'opérateur **new** est un opérateur de haute priorité qui permet d':
 - instancier des objets
 - appeler une méthode particulière de cet objet : le **constructeur**.
- 3- Importation de la classe Personne

Le compilateur cherche le code de la classe Personne dans :

- Le fichier qui contient la méthode principale main()
 - les paquetages importés par les instructions **import**
 - le répertoire à partir duquel le compilateur a été lancé

Java Orienté Objet

(Chapitre 2)



La durée de vie d'un objet

Les objets ne sont pas des éléments statiques

La durée de vie d'un objet passe par 3 étapes :

- la **déclaration** de l'objet et l'**instanciation** grâce à l'opérateur **new**
- l'**utilisation** de l'objet en appelant ses méthodes
- la **suppression** de l'objet : elle est automatique en Java grâce à la machine virtuelle.

Java Orienté Objet

(Chapitre 2)



Objet temporaire :

C'est un objet

- créé par un constructeur
- N'est pas référencé
- construit pour les besoins d'évaluation de l'expression puis abandonné.

l'espace mémoire occupé par cet objet sera automatiquement récupéré ultérieurement

Exemple

```
// import Personne;
public class Test1{
    public static void main(String arg[]){
        new Personne().affiche();
    }
}
```

IHM - Java

15

Java Orienté Objet

(Chapitre 2)



L'objet courant :

- Il est désigné par le mot clé **this** :
 - **this** permet aussi de désigner un attribut de l'objet courant dans lequel se trouve la méthode exécutée :

Exemple :

```
public void initialise(String P, String N, int age){
    this.prenom=P;
    this.nom=N;
    this.age=age;
}
```

L'instruction `this.prenom=P;` → l'attribut prenom de l'objet courant (**this**) reçoit la valeur P.

Remarque

this doit être utilisé explicitement lorsqu'il y a conflit d'identificateurs: Cas de l'instruction :
`this.age=age;`

IHM - Java

16

Java Orienté Objet

(Chapitre 2)



Gestion des objets :

Afficher son type et son emplacement mémoire

```
System.out.println(p1)      Personne@19821f
```

Récupérer son type : méthode "getClass()"

```
p1.getClass();              class Personne
```

Tester son type : opérateur « instanceof » ou mot clé « class »

```
if (p1 instanceof Personne) {...} // true
ou
if (p1.getClass() == Personne.class) {...} // true
```

IHM - Java

17

Java Orienté Objet

(Chapitre 2)



Destruction et ramasse-miettes : Garbage Collectors

La destruction des objets se fait de manière implicite :

Le ramasse-miettes ou Garbage Collector se met en route

Automatiquement :

- Si plus aucune variable ne référence l'objet
- Si le bloc dans lequel il était défini se termine
- Si l'objet a été affecté à "null"

Manuellement :

Sur demande explicite du programmeur " `System.gc()` "

Un pseudo-destructeur « `protected void finalize()` » peut être défini explicitement par le programmeur. Il est appelé juste avant la libération de la mémoire par la machine virtuelle, mais on ne sait pas quand.

IHM - Java

18

Java Orienté Objet

(Chapitre 2)



Les mots clés gérant la visibilité des entités :

Il existe 3 présentés dans un ordre décroissant de niveau de visibilité offert

public : visible par tous les objets

par défaut : visible par les objets des classes se trouvant dans le même package.

protected : visible par les objets de la classe interne ou d'un objet **dérivé**

private : visible uniquement par les objets de la classe interne

En général

les données d'une classe sont déclarées **privées** alors que ses méthodes sont déclarées **publiques**.

Java Orienté Objet

(Chapitre 2)



Encapsulation par l'exemple :

```
public class Personne{
...
    private String nom;
    public void initialise(String P,
String N, int age){
        this.prenom=P;
        this.nom=N;
        this.age=age; }

    public void affiche(){
        System.out.println(nom);}
}
```

```
public class Test1{
    public static void main(String arg[]){
        Personne p1=new Personne ();
        p1.initialise ("Aaaa","Bbbb",30); //Ok
        p1.nom="Cccc";//erreur
        p1.affiche(); //Ok
    }
}
```

Les données (attributs) doivent être protégés et accessibles pour l'extérieur par des **accesseurs**

Java Orienté Objet

(Chapitre 2)



Autres modificateurs :

- **static** : le membre appartient à tous les objets de la classe. Il est inutile d'instancier la classe pour accéder au membre.
- **final** : le membre ne peut pas être modifiée (cas de l'héritage)
- **abstract** : la classe ne pourra pas être instanciée: ses méthodes ne sont pas implémentées

Java Orienté Objet

(Chapitre 2)



Les attributs ou propriétés :

Ce sont des variables :

- variables d'instances

Exemple

```
public class MaClasse { public int valeur1 ; int valeur2 ; protected
int valeur3 ; private int valeur4 ; }
```

- variables de classes

Exemple

```
public class MaClasse { static int compteur ; }
```

- Constantes

Exemple

```
public class MaClasse { final double PI=3.14 ; }
```

Java Orienté Objet

(Chapitre 2)



Initialisation des variables :

En Java

- Toute variable appartenant à un objet est initialisée avec une valeur par défaut
- Cette initialisation ne s'applique pas aux variables locales des méthodes de la classe.
- Les valeurs par défaut lors de l'initialisation automatique des variables d'instances sont :

Type	Valeur par défaut
Boolean	false
byte, short, int, long	0
float, double	0.0
char	\u0000 (null Character)
class	null

IHM - Java

23

Java Orienté Objet

(Chapitre 2)



Les méthodes :

Les **méthodes** sont des fonctions qui implémentent les traitements de la classe.

Syntaxe de déclaration :

```
modificateurs type_val_retour nom_méthode ( arg1, ... ) {
... // définition des variables locales et du bloc d'instructions
}
```

Syntaxe d'appel :

```
Nom_objet.nom_méthode ( par1, ... )
```

Exemples

```
int add(int a, int b) { return a + b; }
```

//Si la méthode retourne un tableau

```
int[] valeurs() { ... } ou int valeurs()[ ] { ... }
```

IHM - Java

24

Java Orienté Objet

(Chapitre 2)



La surcharge :

Définir **plusieurs fois** une même méthode avec des arguments **différents**.

Le compilateur choisi la méthode qui doit être exécutée en fonction des arguments.

→ Simplifier l'interface des classes vis à vis des autres

Exemple classe *Personne*

//première définition

```
public void initialise(String P, String N, int age){
    this.prenom=P;
    this.nom=N;
    this.age=age;
}
```

//Deuxième définition

```
public void initialise(String P, String N){
    this.prenom=P;
    this.nom=N;
}
```

IHM - Java

25

Java Orienté Objet

(Chapitre 2)



Exemple de surcharge :

```
public class Voiture {
    private int puissance;
    private double vitesse;
    private boolean estDemarree;
    //...
    public void accelere(double vitesse) {
        if (estDemarree) {
            this.vitesse = this.vitesse + vitesse;
        }
    }
    public void accelere(int vitesse) {
        if (estDemaree) {
            this.vitesse = this.vitesse +(double)vitesse;
        }
    }
    //...
}
```

```
public class Test1 {
    public static void main (String[] argv) {
        // Déclaration puis création de v1
        Voiture v1 = new Voiture();
        // Accélération 1 avec un double
        v1.accelere(123.5);
        // Accélération 2 avec un entier
        v1.accelere(124);
    }
}
```

IHM - Java

26

Java Orienté Objet

(Chapitre 2)



Les constructeurs :

Un **constructeur** est une méthode qui porte le **nom** de la classe et qui est appelée lors de la **création** de l'objet.

→ Initialiser l'objet

C'est une méthode qui :

- peut accepter des arguments
- ne rend **aucun** résultat.
- Son prototype n'est précédé d'**aucun** type (même **pas** *void*).

Forme générale :

```
nom_classe objet;
```

```
objet=new nom_classe(arg1,arg2, ... argn);
```

ou

```
nom_classe objet =new nom_classe(arg1,arg2, ... argn);
```

IHM - Java

27

Java Orienté Objet

(Chapitre 2)



Les constructeurs :

```
public class Personne{
    private String prenom;
    private String nom;
    private int age;
    // constructeur1 avec parametres
    public Personne(String P, String N, int age){
        this.prenom=P;
        this.nom=N;
        this.age=age;
    }
    // constructeur2 recopie
    public Personne(Personne P){
        this.prenom=P.prenom;
        this.nom=P.nom;
        this.age=P.age;
    }
}
```

NB :
Un constructeur peut être surchargé

IHM - Java

28

Java Orienté Objet

(Chapitre 2)



Les constructeurs :

```
// méthodes
public void initialise(String P, String N, int age){
    this.prenom=P;
    this.nom=N;
    this.age=age;
}

public void affiche(){
    System.out.println(prenom+","+nom+","+age);
}
//...autres méthodes
```

IHM - Java

29

Java Orienté Objet

(Chapitre 2)



Les constructeurs programme test:

```
public class Test1{
    public static void main(String arg[]){

        Personne p1=new Personne("ALAOUI", "Mohammed", 30);
        System.out.print("p1=");
        p1.affiche();

        Personne p2=new Personne(p1);
        System.out.print("p2=");
        p2.affiche();
    }
}
```

```
Console
<terminated> Test [Java Application]
p1=ALAOUI,Mohammed,30
p2=ALAOUI,Mohammed,30
```

IHM - Java

30

Java Orienté Objet

(Chapitre 2)



Programme de test : Les références d'objets :

```
public class Test2{
    public static void main(String arg[]){
        // p1
        Personne p1=new Personne ("ALAOUI", "Mohammed", 30);
        System.out.print("p1=");
        p1.affiche();

        // p2 référence le même objet que p1
        Personne p2=p1;
        System.out.print("p2=");
        p2.affiche();
    }
}
```

```
Console
<terminated> Test [Java Application]
p1=ALAOUI,Mohammed,30
p2=ALAOUI,Mohammed,30
```

IHM - Java

31

Java Orienté Objet

(Chapitre 2)



Programme de test : Les références d'objets :

```
// p3 référence un objet = copie de l'objet réf. par p1
Personne p3=new Personne(p1);
System.out.print("p3=");
p3.affiche();

// on change l'état de l'objet réf. par p1
p1.initialise("ACHIMI", "Fatima", 20);
System.out.print("p1=");
p1.affiche();

// comme p2=p1, l'objet réf. par p2 a du changer d'état
System.out.print("p2=");
p2.affiche();

// comme p3 ne référence pas le même objet que p1
System.out.print("p3=");
p3.affiche();
```

```
Console
<terminated> Test [Java Application] C
p1=ALAOUI,Mohammed,30
p2=ALAOUI,Mohammed,30
p3=ALAOUI,Mohammed,30
p1=ACHIMI,Fatima,20
p2=ACHIMI,Fatima,20
p3=ALAOUI,Mohammed,30
```

IHM - Java

32

Java Orienté Objet

(Chapitre 2)



Passage de paramètres à une méthode :

Un paramètre d'une méthode peut être

- Une variable de type **simple**
- Une **référence** d'un objet typée par n'importe quelle classe

En Java tout est passé **par valeur**

Si les paramètres sont de type **simples** :

- Leur **valeur** est recopiée
- Leur modification dans la méthode **n'entraîne pas** celle de l'original

Si les paramètres sont des objets

- Leur **référence** est recopiée et non pas les attributs
- Leur modification dans la méthode **entraîne** celle de l'original !

IHM - Java

33

Java Orienté Objet

(Chapitre 2)



Exemple de passage :

```
public class param2{
    private static void changeString(String S){
        S="Mardi";
        System.out.println(" Dans la methode S="+S);
    }
    private static void changeInt(int a){
        a=21;
        System.out.println(" Dans la methode a="+a);
    }
    public static void main(String[] arg){
        String S="Lundi";
        changeString(S);
        System.out.println(" Dans main S="+S);
        int jour=20;
        changeInt(jour);
        System.out.println(" Dans main jour="+jour);
    }
}
```

```
Problems @ Javadoc Declaration
<terminated> Prog1 [Java Application] C
Dans la methode S=Mardi
Dans main S=Lundi
Dans la methode a=21
Dans main jour=20
```

IHM - Java

34

Java Orienté Objet

(Chapitre 2)



Exemple de passage :

```
public class Test {
    private static void modifie1(Personne P){
        P.initialise("XXX","YYY",40);
        System.out.print("dans methode1 :");
        P.affiche();
    }
    private static void modifie2(Personne P){
        P=new Personne("AAA","BBB",30);
        System.out.print("dans methode 2 :");P.affiche();
    }
    public static void main(String arg[]){
        Personne p1=new Personne("AAA","BBB",30);
        System.out.print("Dans main : "); p1.affiche();
        modifie1(p1);
        System.out.print("Dans main : "); p1.affiche();
        modifie2(p1);
        System.out.print("Dans main : ");
        p1.affiche();
    }
}
```

```
<terminated> Main (1) [Java Application] C:\P
Dans main : AAA,BBB,30
dans methode1 :XXX,YYY,40
Dans main : XXX,YYY,40
dans methode 2 :AAA,BBB,30
Dans main : XXX,YYY,40
```

IHM - Java

35

Java Orienté Objet

(Chapitre 2)



Accesseurs et Mutateurs (getter/setter):

L'encapsulation permet de sécuriser l'accès aux données privées d'une classe.

La seule façon d'y accéder est d'utiliser les **accesseurs**

Un accesseur est une méthode publique qui donne l'accès à une variable d'instance privée

Exemple :

```
private int valeur = 13;
// accesseur en lecture
public int getValeur(){
    return(valeur);
}
//modificateur
public void setValeur(int val) {
    valeur = val;
}
```

IHM - Java

36

Java Orienté Objet

(Chapitre 2)



Accesseurs et Mutateurs (getter/setter):

Exemple Personne:

```
public class Personne{
    private String prenom;
    private String nom;
    private int age;

    // accesseurs = getters
    public String getPrenom(){ return prenom; }
    public String getNom(){ return nom; }
    public int getAge(){ return age; }

    //modificateurs = setters
    public void setPrenom(String P){ this.prenom=P; }
    public void setNom(String N){ this.nom=N; }
    public void setAge(int age){ this.age=age; }
}
```

IHM - Java

37

Java Orienté Objet

(Chapitre 2)



Accesseurs et Mutateurs (getter/setter):

Exemple Personne:

```
public class Test1{
    public static void main(String[] arg){
        Personne P=new Personne("ALAOUI","Mohammed",20);
        System.out.println("P=("+P.getPrenom()+","+P.getNom()+","+P.getAge()+
            ")");
        P.setAge(24);
        System.out.println("P=("+P.getPrenom()+","+P.getNom()+","+P.getAge()+
            ")");
    }
}

//Résultats :
P=(ALAOUI,Mohammed,20)
P =(ALAOUI,Mohammed,24)
```

IHM - Java

38

Java Orienté Objet

(Chapitre 2)



Les méthodes et attributs de classe :

Attributs :

- Il peut être utile de définir pour une classe des attributs indépendamment des instances : nombre d'objets créées
- Utilisation des Variables de classe comparables aux " variables globales"

Exemple : `private static long nbPersonnes; // nombre de Personnes créées`

Méthodes :

- Ce sont des méthodes qui ne s'intéressent pas à un objet particulier
- Utiles pour des calculs intermédiaires internes à une classe
- Utiles également pour retourner la valeur d'une variable de classe en visibilité *private*
- Elles sont définies comme les méthodes d'instances, mais avec le mot clé **static**

Exemple :

```
public static long getNbPersonnes(){
    return nbPersonnes;
}
```

IHM - Java

39

Java Orienté Objet

(Chapitre 2)



Exemple (static):

```
public class test1{
    public static void main(String arg[]){
        Personne p1=new Personne("ALAOUI", "Mohammed", 30);
        new Personne("HACHIMI", "Fatima", 25);
        Personne p3=p2;
        System.out.println("Nombre de Personnes créées : 
"+Personne.getNbPersonnes());
    }
}
//on obtient les résultats suivants :
Nombre de Personnes créées : 2
```

IHM - Java

40

Java Orienté Objet

(Chapitre 2)



Les tableaux d'objets en Java :

Un objet est une donnée comme une autre : Plusieurs objets peuvent être rassemblés dans un tableau

```
public class test1{
    public static void main(String arg[]){
        Personne[] amis= new Personne[3];
        System.out.println("-----");
        amis[0]=new Personne("ALAOUI", "Mohammed", 30);
        amis[1]=new Personne("HACHIMI", "Fatima", 52);
        amis[2]=new Personne("NOUACH", "Adil", 66);
        int i;
        for(i=0; i<amis.length; i++)
            amis[i].affiche();
    }
}
```

IHM - Java

41

Java Orienté Objet

(Chapitre 2)



L' Héritage :

Définition

Technique offerte par les langages de programmation orientée objet pour construire une classe (ou plusieurs) à partir d'une autre classe (ou plusieurs) en partageant attributs et opérations

Intérêts

Spécialisation, enrichissement : une nouvelle classe réutilise les attributs et les opérations d'une classe en y ajoutant des attributs et/ou des opérations particulières à la nouvelle classe

Redéfinition : une nouvelle classe redéfinit les attributs et opérations d'une classe de manière à en changer le sens et/ou le comportement pour le cas particulier défini par la nouvelle classe

Réutilisation : évite de réécrire du code existant et parfois on ne possède pas les sources de la classe à hériter

IHM - Java

42

Java Orienté Objet

(Chapitre 2)



L' Héritage en Java :

- Utilisation du mot-clé **extends** après le nom de la classe
- Une classe ne peut hériter que **d'une** seule autre classe

Exemple :

```
class Enseignant extends Personne{
    ...}
```

N.B. : Dans certains autres langages (ex : C++) : possibilité d'héritage multiple

- Héritage à plusieurs niveaux

Exemple :

```
class Chef_dep extends Enseignant{
    ...}
```

IHM - Java

43

Java Orienté Objet

(Chapitre 2)



L' Héritage en Java :

Classe *Enseignant* hérite des propriétés de la classe *Personne*,
on écrira :

```
public class Enseignant extends Personne
```

Personne est appelée la **classe mère** (super classe)

- Enseignant la **classe fille** (ou dérivée, sous classe)

Un objet Enseignant a toutes les qualités d'un objet Personne

```
class Enseignant extends Personne{
    // attributs
    private int section;
    // constructeur
    public Enseignant(String P, String N, int age,int section){
        super(P,N,age); //Appel du constructeur de la classe mère
        this.section=section;
    }
}
```

IHM - Java

44

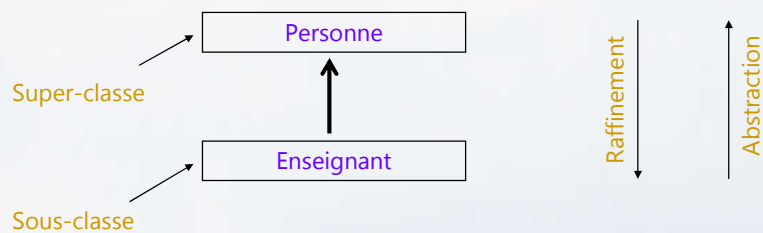
Java Orienté Objet

(Chapitre 2)



L' Héritage en Java :

La généralisation exprime une relation « **est-un** » entre une classe et sa super-classe



L'héritage permet

- de **généraliser** dans le sens abstraction
- de **spécialiser** dans le sens raffinement

IHM - Java

45

Java Orienté Objet

(Chapitre 2)



L' Héritage en Java :

Remarques

- Un objet de la classe **Enseignant** est forcément un objet de la classe **Personne**
 - Un objet de la classe **Personne** n'est pas forcément un objet de la classe **Enseignant**
- Un **Enseignant** est une **Personne**

IHM - Java

46

Java Orienté Objet

(Chapitre 2)



Surcharge et redéfinition :

- **Surcharge** : possibilité de définir des méthodes possédant le même nom mais dont les arguments différents.
- **Redéfinition** : lorsque la sous-classe redéfinit une méthode de la classe mère dont le nom, les paramètres et le type de retour sont identiques

Dans le cas de l'héritage

- Une sous-classe peut ajouter des nouveaux attributs et/ou méthodes à ceux qu'elle hérite (surcharge en fait partie)
- Une sous-classe peut redéfinir des méthodes qu'elle héritent et fournir des implémentations spécifiques pour celles-ci

IHM - Java

47

Java Orienté Objet

(Chapitre 2)



Exemple de Surcharge et redéfinition :

```
public class Personne{
    public String identite(){
        return "Personne("+prenom+", "+nom+", "+age+")";
    }
}

class Enseignant extends Personne{
    int sec;
    public String identite(){
        return "Enseignant("+super.identite()+", "+sec+")";
    }
}
```

IHM - Java

48

Java Orienté Objet

(Chapitre 2)



En général :

si O est un objet et M une méthode

pour exécuter la méthode $O.M$, le système cherche une méthode M dans l'ordre suivant :

- dans la classe de l'objet O
- dans sa classe mère s'il en a une
- dans la classe mère de sa classe mère si elle existe
- etc...

L'héritage permet donc de redéfinir dans la classe fille des méthodes de même nom dans la classe mère

→ Adapter la classe fille à ses propres besoins.

Java Orienté Objet

(Chapitre 2)



Polymorphisme et Java : Surclassement :

Java permet le polymorphisme

- A une référence déclarée d'une classe (**Personne**), il est possible d'affecter une référence vers un objet de sa *sous classe* (**Enseignant**)
- On parle de **surclassement** d'objets ou **upcasting**
- Plus généralement, à une référence d'un type donné, soit A , il est possible d'affecter une valeur qui correspond à une référence vers un objet dont le type effectif est n'importe quelle sous classe directe ou indirecte de A

Java Orienté Objet

(Chapitre 2)



Exemple 1:

```
public class Test {
    public static void main (String[] argv) {

        Personne P1 = new Enseignant("Aaaa", "Bbbb", 50, 10); //OK

        P1.setSec(8); //ERROR

        System.out.println(P1.identite()); //OK
    }
}
```

IHM - Java

51

Java Orienté Objet

(Chapitre 2)



Exemple 2:

```
public class test1{
    public static void main(String arg[]){
        Enseignant e= new Enseignant("ALAOUI", "Mohammed", 56, 11);
        affiche(e);

        Personne p= new Personne("HACHIMI", "Fatima", 30);
        affiche(p);
    }
    public static void affiche(Object obj){
        System.out.println(obj.toString());
    }
}
```

Les résultats obtenus sont les suivants :

Enseignant@1ee789

Personne@1ee770

IHM - Java

52

Java Orienté Objet

(Chapitre 2)



Procédure :

Le système devra exécuter l'instruction `System.out.println(e.toString())` où `e` est un objet `Enseignant`.

Il cherche une méthode `toString()` dans la hiérarchie des classes menant à la classe **`Enseignant`** en commençant par la dernière :

1. dans la classe **`Enseignant`**, il ne trouve pas de méthode `toString()`
2. dans la classe mère **`Personne`**, il ne trouve pas de méthode `toString()`
3. dans la classe mère **`Object`**, il trouve la méthode `toString()` et l'exécute

Java Orienté Objet

(Chapitre 2)



Remarque :

On peut redéfinir la méthode `toString` de la classe mère `Object` pour les classes `Personne` et `Enseignant`.

```
public class Personne{
    //...
    public String toString(){
        return "Personne("+prenom+", "+nom+", "+age+")";
    }
}
class Enseignant extends Personne{
    private int section;
    //...
    public String toString(){
        return "Enseignant("+super.toString()+", "+section+")";
    }
}
```

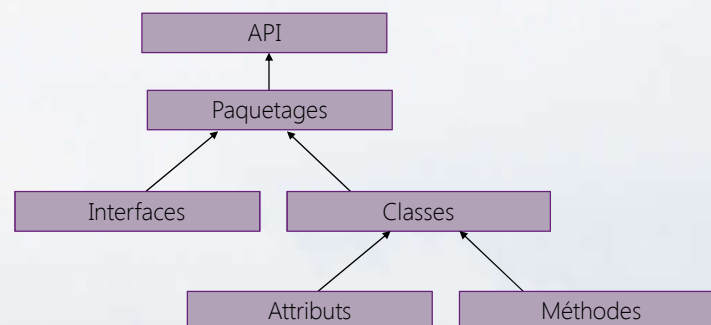
Java Orienté Objet

(Chapitre 2)



paquetages (packages) :

- Le langage Java propose une définition très claire du mécanisme d'empaquetage qui permet de classer et de gérer les API externes
- Les API sont constituées :



IHM - Java

55

Java Orienté Objet

(Chapitre 2)



paquetages (packages) :

Un paquetage est donc un **groupe de classes** associées à une fonctionnalité

- **java.io** : lecture et écriture
- **java.lang** : rassemble les classes de base Java (**Object**, **String**, **System**, ...)
- **java.util** : rassemble les classes utilitaires (**Collections**, **Date**, ...)

...

L'utilisation des paquetages permet de regrouper les classes afin d'organiser des libraires de classes Java

IHM - Java

56

Java Orienté Objet

(Chapitre 2)



Les paquetages : Utilisation des classes :

- Lorsque, dans un programme, il y a une référence à une classe, le compilateur la recherche dans le paquetage par défaut (**java.lang**)
- Pour les autres, il est nécessaire de fournir explicitement l'information pour savoir où se trouve la classe :

Utilisation d'**import** Nom du paquetage avec le nom de la classe

Exemples:

```
import mesclasses.Point;
import java.lang.String; // Ne sert à rien puisque par défaut
import java.io.ObjectOutput;
import mesclasses.*;
import java.lang.*; // Ne sert à rien puisque par défaut
import java.io.*;
java.io.ObjectOutput toto = new java.io.ObjectOutput(...)
```

IHM - Java

57

Java Orienté Objet

(Chapitre 2)



Les paquetages : leur « existence » physique :

A chaque classe Java correspond un **fichier**

A chaque paquetage (sous- paquetage) correspond un **répertoire**

Un paquetage peut contenir

- Des classes ou des interfaces
- Un autre paquetage (sous- paquetage)

Exemple :



IHM - Java

58

Java Orienté Objet

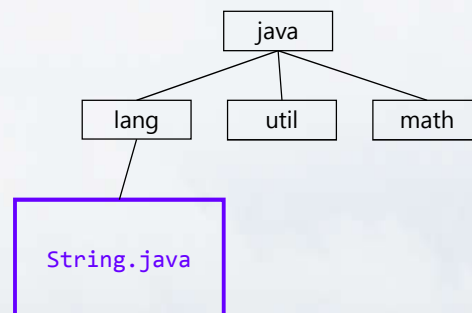
(Chapitre 2)



Les paquets : hiérarchie de paquets :

A une hiérarchie de paquetage correspond une hiérarchie de répertoires dont les noms coïncident avec les composants des noms de paquetage

Exemple : la classe **String**



IHM - Java

59

Java Orienté Objet

(Chapitre 2)



Les paquets : visibilité :

- L'instruction **import nomPackage.*** ; ne concerne que les classes du paquetage indiqué.
- Elle ne s'applique pas aux classes des sous-paquetages

```

import java.util.zip.*;
import java.util.*;
public class Essai {
    ...
    public Essai() {
        Date d = new Date(...);
        ZipFile z = new ZipFile(...);
        ...
    }
    ...
}
  
```

Paquetages
différents

IHM - Java

60

Java Orienté Objet

(Chapitre 2)



Classes internes :

C'est une classe définit **au sein d'une autre** classe

L'utilisation d'une classe interne est souhaitable lorsque la classe n'a d'utilité que dans la classe qui la contient.

Soit **Article** une classe interne de la classe **Test**

Lors de la compilation du source **Test.java** on obtient deux fichiers .class :

1. **Test\$Article.class**
2. **Test.class**

Java Orienté Objet

(Chapitre 2)



Exemple : Classe interne article de la classe Test :

<pre>//classes importées import java.io.*; public class Test{ //classe interne private class Article{ //on définit la structure private String code; private String nom; private double prix; //constructeur public Article(String code, String nom, double prix){ this.code=code; this.nom=nom; this.prix=prix; } } //fin classe article</pre>	<pre>//données locales private Article art; //constructeur public Test(String code, String nom, double prix){ art= new Article(code, nom, prix);} //accesseur public Article getArticle(){ return art;} //toString public String toString(){ return "Article("+art.code+","+art.nom+","+art.prix+"") ";}</pre>
---	---

Java Orienté Objet

(Chapitre 2)



Exemple : Classe interne article de la classe Test :

```
//main
public static void main(String arg[]){
    Test t1= new Test("a100","velo",1000);
    System.out.println("art="+t1.getArticle());
    System.out.println("art="+t1.toString());
}
} // fin classe Test
```

```
<terminated> Test (1) [Java Application] C:\Program Files\Java
art=Test$Article@2a139a55
art=Article (a100,velo,1000.0)
```

IHM - Java

63

Java Orienté Objet

(Chapitre 2)



Classes et méthodes abstraites

On ne connaît pas **toujours** le **comportement** par défaut d'une opération commune à plusieurs sous-classes

Exemple : aire d'une forme géométrique. On sait que toutes les formes géométriques ont une aire, mais la formule est différente d'une forme à l'autre

Solution : on peut déclarer la méthode « **abstraite** » dans la classe mère et ne pas lui donner d'implémentation par défaut

Méthode abstraite et conséquences : 3 règles **à retenir**

- 1- Si **une seule** des méthodes d'une classe est **abstraite**, alors la classe devient aussi abstraite
- 2- On ne peut **pas** instancier une classe abstraite car au moins une de ses méthodes n'a pas d'implémentation
- 3- Toutes les classes filles héritant de la classe mère abstraite doivent implémenter **toutes** ses méthodes abstraites ou sinon elles sont aussi abstraites

IHM - Java

64

Java Orienté Objet

(Chapitre 2)



Classes abstraites et Java :

On utilise le mot clé **abstract** pour spécifier une classe abstraite

Une classe abstraite se déclare ainsi :

```
public abstract class NomMaClasse {
    ...
}
```

Une méthode abstraite se déclare ainsi :

```
public abstract void maMethode(...);
```

NB :

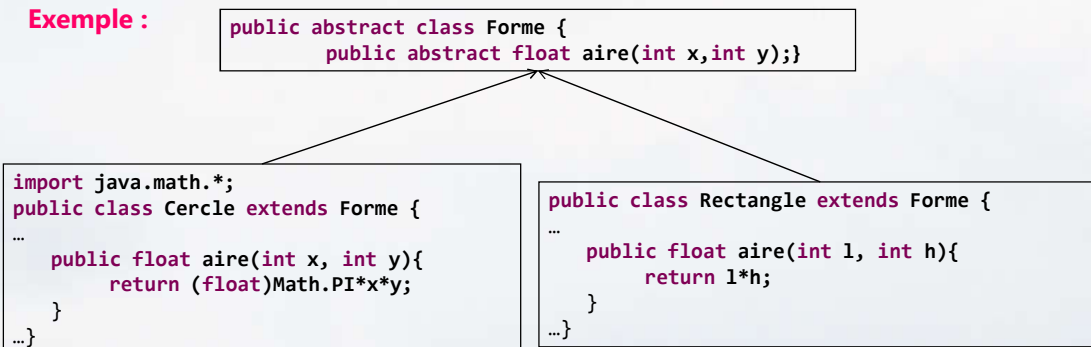
Pour créer une méthode abstraite, on déclare sa signature (nom et paramètres) sans spécifier le corps et en ajoutant le mot-clé **abstract**

Java Orienté Objet

(Chapitre 2)



Exemple :



Attention : ce n'est pas de la redéfinition. On parle d'implémentation de méthode abstraite

Java Orienté Objet

(Chapitre 2)



Programme de test :

```
public class test {
    public static void main(String[] args) {
        ...
        Cercle c1=new Cercle(1,1,25);
        System.out.println("L'aire est "+c1.aire(25, 25));
        ...
        Forme f1=new forme(); // Erreur
    }
}
```

Attention : La classe Forme ne peut être instanciée puisqu'elle est abstraite

IHM - Java

67

Java Orienté Objet

(Chapitre 2)



Interfaces :

Une interface est un **modèle** pour une classe

- Quand **toutes les méthodes** d'une classe sont **abstraites** et il n'y a aucun attribut on aboutit à la notion d'interface
- Elle définit la **signature** des méthodes qui doivent être implémentées dans les classes qui respectent ce modèle (Ce contrat)
- Toute classe qui implémente l'interface doit implémenter **toutes** les méthodes définies par l'interface
- Tout objet instance d'une classe qui implémente l'interface peut être déclaré comme étant du **type** de cette interface
- Les interfaces pourront se **dériver**

IHM - Java

68

Java Orienté Objet

(Chapitre 2)



Mise en œuvre d'une interface :

- La définition d'une interface se présente comme celle d'une classe. On y utilise simplement le mot clé **interface** à la place de **class**

```
public interface NomInterface {
    ...}
```

- Lorsqu'on définit une classe, on peut préciser qu'elle implémente une ou plusieurs interfaces donnée(s) en utilisant une fois le mot clé **implements**

```
public class NomClasses implements Interface1, Interface2, ... {
    ...}
```

- Si une classe hérite d'une autre classe elle peut également implémenter une ou plusieurs interfaces

```
public class NomClasses extends SuperClasse implements Interface1, ...
{
    ...}
```

IHM - Java

69

Java Orienté Objet

(Chapitre 2)



Mise en œuvre d'une interface :

- Une interface ne possède **pas** d'attribut
- Une interface peut posséder des **constantes**
- Une interface ne possède pas de mot clé **abstract**
- Les interfaces ne sont **pas instanciables** (Même raisonnement avec les classes abstraites)

```
public interface NomInterface {
    public static final int CONST = 2; }
```

```
NomInterface jeTente = new NomInterface(); // Erreur!!
```

IHM - Java

70

Java Orienté Objet

(Chapitre 2)



Exemple : Mise en œuvre d'une interface :

```
//une interface I1
public interface I1{
    int ajouter(int i,int j);
    int soustraire(int i,int j);
}
```

```
public class C1 implements I1 {
    public int ajouter(int i,int j) {
        return i+j+1;
    }
    public int soustraire(int i,int j) {
        return i-j+1;
    }
}
```

```
public class C2 implements I1 {
    public int ajouter(int i,int j) {
        return i+j+10;
    }
    public int soustraire(int i,int j) {
        return i-j+10;
    }
}
```

IHM - Java

71

Java Orienté Objet

(Chapitre 2)



Exemple : Mise en œuvre d'une interface :

Tout objet instance d'une classe qui implémente l'interface peut être déclaré comme étant du **type de cette interface**

Exemple :

```
public class Test{
    ...
    public static void main(String[] arg){
        // création d'un objet C
        C1 c= new C1();
        ...
    }//main
}//classe test
```

```
I1 c= new C1();
```

IHM - Java

72

Java Orienté Objet

(Chapitre 2)



Mise en œuvre d'une interface :

Les interfaces pourront se dériver

- Une interface peut hériter d'une autre interface : `extends`

```
public interface I2 extends I1{
    ...}
```

Conséquences

- La définition de méthodes de l'interface mère I1 sont reprises dans l'interface fille I2
- Toute classe qui implémente l'interface fille **doit** donner une implémentation à toutes les méthodes mêmes celle héritées

Utilisation

- Lorsqu'un modèle peut se définir en plusieurs sous-modèles complémentaires

IHM - Java

73

Java Orienté Objet

(Chapitre 2)



Programme de test : Importance des interfaces :

```
public class Test{
    private static void calculer(int i, int j, I1 in){
        System.out.println(in.ajouter(i,j));
        System.out.println(in.soustraire(i,j));
    }
    public static void main(String[] arg){
        // création de deux objets C1 et C2
        C1 c1= new C1();
        C2 c2= new C2();
        // appels de la fonction statique calculer
        calculer(4,3,c1);
        calculer(14,13,c2);
    }
}
```

IHM - Java

74

Java Orienté Objet

(Chapitre 2)



Exercice :

Dans cet exercice, on manipule des formes géométriques que l'on définit par l'interface suivante:

```
interface FormeGeometrique {
    double perimetre();
    double surface();
}
```

Écrire par les classes **Cercle**, **Triangle**, **Rectangle**, **Carre**.

Un **polygone** est la forme définie par une ligne brisée

- qui ne se coupe pas elle-même ;
- qui commence et termine au même point.

Quel sera le type de Polygone ? Quelle sera sa relation avec les classes précédentes ?

Java Orienté Objet

(Chapitre 2)



Exceptions :

Définition

Une **exception** est un signal qui indique que quelque chose d'exceptionnel (comme une erreur) s'est produite. Elle interrompt le flot d'exécution normal du programme

▪ A quoi sert la gestion des exceptions?

- Gérer les erreurs est indispensable : **Mauvaise gestion** peut avoir des conséquences graves
- Mécanisme simple et lisible : **Regroupement** du code réservé au traitement des erreurs
- Possibilité de « **recupérer** » une erreur à plusieurs niveaux d'une application (propagation dans la pile des appels de méthodes)

▪ Vocabulaire

- détecter et traiter les erreurs (**try** **catch** **finally**)
- lever ou propager (**throw** **throws**)

Java Orienté Objet

(Chapitre 2)



Exceptions :

Mécanisme de traitement:

En java, on peut classer les erreurs en deux catégories :

- Les erreurs **surveillées**
Le programmeur est obligé de les traiter
- Les erreurs **non surveillés**
Considérées trop graves pour que leur traitement soit prévu à priori

Lorsqu'une erreur est rencontrée, l'interpréteur :

- Crée immédiatement un objet instance d'une classe particulière et sous classe de la classe **Exception**
- Cherche le code susceptible de la traiter avec le bloc **try catch**

IHM - Java

77

Java Orienté Objet

(Chapitre 2)



Exemple : Interceptor une erreur :

```
public class Test{
    public static void main(String[] args){
        int[] tab=new int[] {0,1,2,3}; // déclaration & initialisation
        int i;
        for (i=0; i<tab.length; i++)
            System.out.println("tab[" + i + "]= " + tab[i]);
        tab[100]=6;
    } //main
} //class
```

```
<terminated> Prog1 [Java Application] C:\Program Files\Java\jre1.8.0_60\bin\javaw.exe (23 janv. 2016 19:26:31)
tab[0]=0
tab[1]=1
tab[2]=2
tab[3]=3
Exception in thread "main" java.lang.ArrayIndexOutOfBoundsException: 100
    at Prog1.main(Prog1.java:9)
```

IHM - Java

78

Java Orienté Objet

(Chapitre 2)



Le bloc try catch finally :

```
try {
    operation_risquée1; opération_risquée2;
} catch (ExceptionInteressante e) {
    traitements
} catch (ExceptionParticulière e) {
    traitements
} catch (Exception e) {
    traitements
} finally{
    traitement_pour_terminer_proprement;
}
```

NB:

- Le bloc **Finally** est toujours exécuté, qu'une exception soit levée ou non
- Dans l'ordre séquentiel des clauses **catch**, un type d'exception de classe ne doit pas venir après un type d'une exception d'une super classe.

IHM - Java

81

Java Orienté Objet

(Chapitre 2)



Lancer une exception :

Une exception peut être lancée ou propagée.

Une méthode peut déclarer qu'elle lance une exception par le mot clé **throws**

```
public static void paiement(int age) throws Exception {
    ...}
```

Permet à la méthode calcul de propager une exception de type Exception

```
public static void paiement(int age) throws Exception {
    if(age<=0) throw new Exception("L'âge doit être positive");
    if(age<18)System.out.println("vous allez payer 100");
    else System.out.println("vous allez payer 300");
}
```

IHM - Java

82

Java Orienté Objet

(Chapitre 2)



Exemple:

```
public class Test {
    public static void paiement(int age) throws Exception {
        if(age<=0) throw new Exception("L'âge doit être positive");
        if(age<18) System.out.println("IL faut payé 100 DH");
        else System.out.println("IL faut payé 300 DH"); }

    public static void main(String[] args) {
        Scanner s=new Scanner(System.in);
        System.out.println("Saisir votre age");
        int a=0;
        a=s.nextInt();
        s.close();
        try {paiement(a);}
        catch (Exception e) {System.err.println(e.getMessage());}
        finally{System.out.println("Programme terminé");};
    }
}
```

```
<terminated> Test [Java Application]
Saisir votre age
0
L'âge doit être positive
Programme terminé
```

IHM - Java

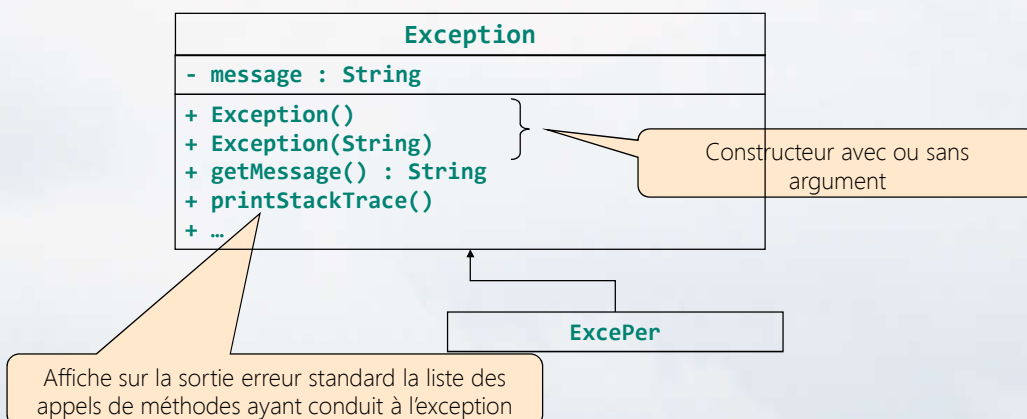
83

Java Orienté Objet

(Chapitre 2)

Créer ses propres exceptions

Les exceptions sont des objets, nous pouvons donc étendre les classes d'exceptions



IHM - Java

84

Java Orienté Objet

(Chapitre 2)

Créer ses propres exceptions

```
class ExcePer extends ArrayIndexOutOfBoundsException{
    ExcePer(String s){
        super(s);
    }
}
```

```
public class Test{
    public static void main(String[] args){
        int[] tab=new int[] {0,1,2,3};
        for (int i=0; i<6; i++)
            if(i>=tab.length){throw new ExcePer(" ATTENTION la valeur de l'indice
            n'est pas possible");}
            else System.out.println("tab[" + i + "]= " + tab[i]);
        } //main
    } //class
```

```
<terminated> Test [Java Application] C:\Program Files\Java\jre1.8.0_241\bin\javaw.exe (6 mars 2021 13:51:11)
tab[0]=0
tab[1]=1
tab[2]=2
tab[3]=3
Exception in thread "main" ExcePer:  ATTENTION la valeur de l'indice n'est pas possible
at Test.main(Test.java:18)
```

IHM - Java

5

Java Orienté Objet

(Chapitre 2)

Les exceptions courantes

- Java fournit de nombreuses classes prédéfinies dérivées de la classe **Exception**
 - **ArithmeticException** (division par zéro)
 - **NullPointerException** (référence non construite)
 - **ClassCastException** (problème de cast)
 - **IndexOutOfBoundsException** (problème de dépassement d'index dans tableau)

IHM - Java

86